# Практическая работа №4

---

## Docker Compose: оркестрация сервисов с Redis-кэшированием

---

Персональные параметры (заполните **ДО** начала работы):

| Параметр | Как определить | Ваше значение |
|---|---|---|
| Номер ИСУ | Можно взять из ЛК | — |
| Персональный порт (app) | 5430 + последние 2 цифры ИСУ<br>Пример:<br>5430 + 45 = 5475 | — |
| Персональный порт (Redis) | 6370 + последние 2 цифры ИСУ<br>Пример:<br>6370 + 45 = 6415 | — |
| Персональный префикс | Фамилия латиницей в нижнем регистре (например, ivanov) | — |
| Секретный ключ | secret+последние 2 цифры ИСУ+фамилия на английском | — |
| Имя базы данных | фамилия+последние 2 цифры ИСУ | — |

**!!!Важно!!!** Все команды и файлы должны использовать ваши персональные значения. Отчёты с портами по умолчанию (5432, 6379, 8000) или без префикса <фамилия> — автоматически получают 0 баллов за соответствующие этапы.

**Этап 1. Подготовка проекта для Docker Compose**

Откройте репозиторий из предыдущих лабораторных на своём ПК. Скопируйте ветку lab-3

Задание 1.1 — Создайте файл .env с персональными переменными

В корне проекта создайте файл .env. Он будет содержать все персональные переменные, которые docker-compose подставит в сервисы автоматически.

```
# .env — НЕ коммитьте этот файл в git!
APP_OWNER=<ваша_фамилия>
APP_SECRET=<ваш_секретный_ключ>
APP_PORT=<ваш_порт_app>
POSTGRES_USER=user_<цифры>
POSTGRES_PASSWORD=pgpass_<цифры>
POSTGRES_DB=assistant_<цифры>
PG_PORT=<ваш_порт_postgres>
REDIS_PORT=<ваш_порт_redis>
CACHE_TTL=120
```

Задание 1.2 — Создайте docker-compose.yml с сервисами app и db

Создайте файл docker-compose.yml в корне проекта. На данном этапе опишите два сервиса: PostgreSQL (db) и приложение (app). Используйте переменные из .env.

```
# docker-compose.yml
version: '3.8'
services:
  db: тут будет описания вашей бд
  app: тут будет описания вашего сервиса
volumes:
  <фамилия>-pgdata:
networks:  <фамилия>-backend:
    driver: bridge
```

Обратите внимание желательно указать depends_on с condition, таким образом благодаря service_healthy приложение не запустится, пока

PostgreSQL не пройдёт healthcheck. Это гарантирует корректный порядок запуска.

Задание 1.3 — Проверьте базовый запуск

```
# Запуск всех сервисов
docker-compose up -d --build

# Проверка статуса
docker-compose ps

# Проверка логов
docker-compose logs app
docker-compose logs db
```

Убедитесь, что оба контейнера запущены и приложение отвечает:

```
curl http://localhost:<ваш_порт_app>/
curl http://localhost:<ваш_порт_app>/health
```

Скопируйте в отчёт скриншот с выводом docker-compose ps и обоих curl. На скриншоте должны быть видны персональные порты и имя владельца.

**Этап 2. Добавление Redis для кэширования погоды**

В этом этапе вы добавите сервис Redis в docker-compose.yml и модифицируете приложение так, чтобы эндпоинт /weather/{city} кэшировал результаты. Это предотвратит спам внешнего API при повторных запросах.

Задание 2.1 — Добавьте сервис Redis в docker-compose.yml

Добавьте новый сервис redis в docker-compose.yml. Приложение (app) должно зависеть от Redis.

```
 redis:
   image: redis:7-alpine
   container_name: <фамилия>-redis
   ports:
     - "${REDIS_PORT}:6379"
   networks:
     - <фамилия>-backend
   healthcheck:
     test: ["CMD", "redis-cli", "ping"]
     interval: 5s
     timeout: 3s
     retries: 5
```

Также обновите секцию depends_on у сервиса app, добавив зависимость от redis:

Задание 2.3 — кэширование в приложении

Убедитесь что в main.py интегрирован redis

```
import redis
import json

REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")
CACHE_TTL = int(os.getenv("CACHE_TTL", "120"))
redis_client = redis.from_url(REDIS_URL, decode_responses=True)
```

Проверьте что эндпоинт /weather/{city}, имеет логику кэширования.

```
@app.get("/weather/{city}")
async def weather(city: str):
    cache_key = f"weather:{OWNER}:{city.lower()}"

    # Проверяем кэш
    cached = redis_client.get(cache_key)
    if cached:
        data = json.loads(cached)
        data["source"] = "cache"
        return data

    #  текущий код

    # ... получение weather_data ...

    result = {
        "city": city_name,
        "country": country,
        "temperature_c": weather_data["temperature_2m"],
        "humidity_pct": weather_data["relative_humidity_2m"],
        "wind_km_h": weather_data["wind_speed_10m"],
        "condition": WEATHER_CODES.get(code, f"Code {code}"),
        "source": "api"
    }

    # Сохраняем в кэш
    redis_client.setex(cache_key, CACHE_TTL, json.dumps(result))
    return result
```

**Этап 3. Запуск и проверка полного стека**

Задание 3.1 — Пересоберите и запустите все сервисы

На скриншоте docker-compose ps должны быть видны три сервиса: <фамилия>-postgres, <фамилия>-assistant, <фамилия>-redis — все в статусе Up (healthy).

Задание 3.2 — Проверьте порядок запуска (depends_on)

```
# Просмотрите логи — обратите внимание на порядок запуска
docker-compose logs --tail=30

# Убедитесь, что app стартовал после db и redis
docker-compose logs app | head -5
```

Скопируйте в отчёт скриншот логов, на котором видно, что сервисы стартовали в правильном порядке (db и redis перед app).

Задание 3.3 — Проверьте работу кэширования

```
# Первый запрос — данные придут от API
curl http://localhost:<ваш_порт_app>/weather/Moscow

# Второй запрос — данные должны прийти из кэша
curl http://localhost:<ваш_порт_app>/weather/Moscow

# Проверьте статистику кэша
curl http://localhost:<ваш_порт_app>/cache/stats

# Проверьте данные напрямую в Redis
docker exec <фамилия>-redis redis-cli KEYS "weather:*"
docker exec <фамилия>-redis redis-cli TTL
weather:<ваша_фамилия>:moscow
```

Скопируйте в отчёт скриншот, на котором видно:
— первый запрос с "source": "api"
— второй запрос с "source": "cache"
— вывод /cache/stats с вашей фамилией и кэшированными городами

Задание 3.4 — Исследуйте networks и volumes

```
# Список сетей
docker network ls --filter "name=<фамилия>"

# Детали сети — должны быть видны все 3 контейнера
docker network inspect <фамилия>-backend

# Список volumes
docker volume ls --filter "name=<фамилия>"
```

Скопируйте в отчёт скриншот docker network inspect, на котором видны все три контейнера в одной сети.

**Этап 4. Масштабирование сервиса**

Docker Compose позволяет масштабировать сервисы — запускать несколько экземпляров одного контейнера. Для этого необходимо убрать фиксированные container_name и ports с конкретным хостовым портом у масштабируемого сервиса.

Задание 4.1 — Подготовьте сервис app к масштабированию

Создайте отдельный файл docker-compose.scale.yml (или измените основной). Для масштабирования уберите container_name у app и измените порт на диапазон:

```
# docker-compose.scale.yml
version: '3.8'

services:
  db:
    image: postgres:15
    container_name: <фамилия>-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - <фамилия>-pgdata:/var/lib/postgresql/data
    networks:
      - <фамилия>-backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: <фамилия>-redis
    networks:
      - <фамилия>-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    # container_name убран для масштабирования!
    ports:
      - "8000"  # Docker назначит случайный хостовый порт
    environment:
      APP_OWNER: ${APP_OWNER}
      APP_SECRET: ${APP_SECRET}
      DATABASE_URL:
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432
/${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
      CACHE_TTL: ${CACHE_TTL}
    depends_on:
      db:
```

```
      condition: service_healthy
    redis:
      condition: service_healthy
  networks:
    - <фамилия>-backend

volumes:
 <фамилия>-pgdata:

networks:
 <фамилия>-backend:
  driver: bridge
```

Задание 4.2 — Запустите масштабирование

```
# Остановите текущие контейнеры
docker-compose down

# Запустите с масштабированием (3 экземпляра app)
docker-compose -f docker-compose.scale.yml up -d --build --scale app=3

# Проверьте, что запущено 3 экземпляра
docker-compose -f docker-compose.scale.yml ps
```

Скопируйте в отчёт скриншот docker-compose ps, на котором видны 3 экземпляра app с разными портами, а также db и redis.

Задание 4.3 — Проверьте работу масштабированных экземпляров

```
# Узнайте назначенные порты
docker-compose -f docker-compose.scale.yml ps app

# Обратитесь к каждому экземпляру (подставьте реальные порты из вывода выше)
curl http://localhost:<порт_1>/health
curl http://localhost:<порт_2>/health
curl http://localhost:<порт_3>/health

# Проверьте, что кэш Redis общий для всех экземпляров
curl http://localhost:<порт_1>/weather/London
curl http://localhost:<порт_2>/weather/London
curl http://localhost:<порт_3>/cache/stats
```

Скопируйте в отчёт скриншот, показывающий, что все три экземпляра отвечают на /health и используют общий Redis-кэш (второй и третий запрос к /weather/London должны вернуть "source": "cache").

Задание 4.4 — Вернитесь к основной конфигурации

```
# Остановите масштабированные контейнеры
docker-compose -f docker-compose.scale.yml down

# Запустите основную конфигурацию
docker-compose up -d
```

**Этап 5. Финальный артефакт**

Задание 5.1 — Финальная проверка всего стека

```
echo "========================================" && \
echo "ЛР4 завершена: $(date '+%d.%m.%Y %H:%M:%S')" && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "========================================" && \
echo "" && \
echo "Docker Compose сервисы:" && \
docker-compose ps && \
echo "" && \
echo "Сети:" && \
docker network ls --filter "name=<фамилия>" && \
echo "" && \
echo "Volumes:" && \
docker volume ls --filter "name=<фамилия>" && \
echo "" && \
echo "Кэш Redis:" && \
docker exec <фамилия>-redis redis-cli KEYS "weather:*" && \
echo "" && \
echo "Погода (из кэша):" && \
curl -s http://localhost:<ваш_порт_app>/cache/stats | python3 -m json.tool
```

Скриншот полного вывода этой команды. Должны быть видны:

Три контейнера (<фамилия>-postgres, <фамилия>-assistant, <фамилия>-redis)

Именованный volume (<фамилия>-pgdata)

Пользовательская сеть (<фамилия>-backend)

Данные кэша с вашей фамилией

Логин и хост студента, время выполнения

**Требования к отчёту**

**Файл должен называться**: ПР1_Фамилия_ИСУ.pdf (например, ПР1_Иванов_4123.pdf)

**Обязательные разделы:**

Заполненная таблица персональных параметров

**Скриншоты:**

Этап 1.3: docker-compose ps + curl с персональными портами

Этап 3.2: логи с порядком запуска сервисов

Этап 3.3: кэширование — первый запрос (api), второй запрос (cache), /cache/stats

Этап 3.4: docker network inspect с тремя контейнерами

Этап 4.2: docker-compose ps с 3 экземплярами app

Этап 5.1: финальный артефакт с логином/хостом/временем и всей инфраструктурой